AppExampleKafka — API de Autenticação

Clean Architecture · JWT Bearer · Refresh Token · Cookie HTTP-only

Unit of Work · Repository · AutoMapper · FluentValidation · EF Core + PostgreSQL

Plataforma	.NET 10 · ASP.NET Core (Controllers)
Banco de dados	PostgreSQL (Npgsql + EF Core 10)
Autenticação	Access token JWT (curto) + Refresh token rotacionável
Transporte do refresh	Cookie HTTP-only, Secure, SameSite=Strict
Documento gerado em	10/09/2026

1. Visão Geral

Esta API implementa um fluxo de autenticação completo e seguro com apenas duas roles — **Admin** e **User** — seguindo os princípios da **Clean Architecture**. O modelo de tokens combina dois elementos complementares:

- **Access token (JWT)**: curta duração (60 min). Enviado no header `Authorization: Bearer <token>` e validado sem consulta ao banco (stateless).
- **Refresh token**: longa duração (7 dias), opaco e aleatório (256 bits). Trafega exclusivamente em um **cookie HTTP-only**, é persistido apenas como **hash SHA-256** e é **rotacionado** a cada uso.

Essa separação é a prática recomendada: o access token, por ser curto, limita a janela de exposição caso vaze; o refresh token, por ficar em cookie HTTP-only, é inacessível a JavaScript, mitigando o roubo de credenciais via XSS.

1.1. Por que cookie HTTP-only para o refresh token?

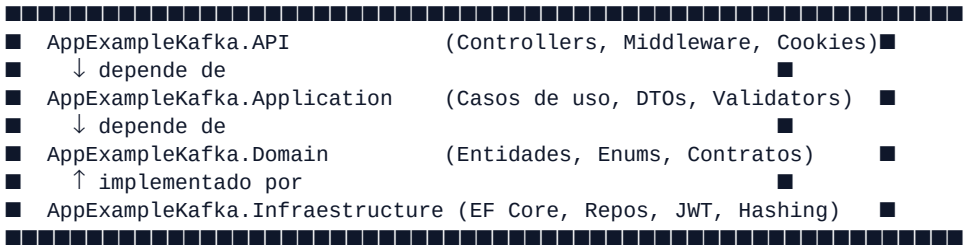
- **HttpOnly**: o cookie não é acessível via `document.cookie`. Mesmo que um script malicioso seja injetado (XSS), ele não consegue ler o refresh token.
- **Secure**: o cookie só é enviado sob HTTPS, impedindo interceptação em texto claro.
- **SameSite=Strict**: o navegador não envia o cookie em requisições cross-site, mitigando ataques CSRF.
- O **access token**, por outro lado, fica com o cliente (memória) e é enviado manualmente no header — nunca em cookie — evitando o envio automático que abriria brecha de CSRF.

1.2. Rotação de refresh token

A cada chamada a `/refresh`, o token usado é **revogado** e um **novo** é emitido (com registro de qual token o substituiu). Isso significa que um refresh token só pode ser usado uma vez. Se um token já usado (revogado) for apresentado novamente, a requisição é rejeitada com **401** — sinal clássico de reutilização indevida.

2. Arquitetura em Camadas

A dependência sempre aponta para dentro: camadas externas conhecem as internas, nunca o contrário. O Domain não referencia nenhuma outra camada.



A Infraestrutura implementa as interfaces definidas no Domain (Repositories, Unit of Work) e na Application (ITokenService, IPasswordHasher), sendo conectada por injeção de dependência na inicialização — o núcleo permanece desacoplado de framework e banco.

2.1. Estrutura de pastas (arquivos relevantes)

```
AppExampleKafka.Domain
  Entities/         User.cs, RefreshToken.cs
  Enums/            UserRole.cs
  Repositories/     IUserRepository, IRefreshTokenRepository, IUnitOfWork

AppExampleKafka.Application
  DTOs/Auth/        RegisterRequest, LoginRequest, UserDto,
                    AuthResponse, AuthResult
  Common/Interfaces IAuthService, ITokenService, IPasswordHasher
  Common/Exceptions AppException (400/401/409)
  Validators/       RegisterRequestValidator, LoginRequestValidator
  Mappings/         UserProfile (AutoMapper)
  Services/         AuthService

AppExampleKafka.Infraestructure
  Context/          AppDbContext, Configurations/, Migrations/
  Repositories/     UserRepository, RefreshTokenRepository
  UnitOfWork/       UnitOfWork
  Security/         PasswordHasher, TokenService, JwtSettings

AppExampleKafka.API
  Controllers/      AuthController
  Middleware/       ExceptionHandlingMiddleware
  Program.cs, appsettings.json
```

3. Camada de Domínio (Domain)

Entidades ricas com setters privados: o estado só muda por construtor ou por métodos de negócio explícitos. Não há dependência de framework.

3.1. Entidade RefreshToken

```
public class RefreshToken
{
    public Guid Id { get; private set; } = Guid.NewGuid();
    public Guid UserId { get; private set; }
    public string TokenHash { get; private set; }      // SHA-256, nunca o valor bruto
    public DateTime ExpiresAtUtc { get; private set; }
    public DateTime CreatedAtUtc { get; private set; }
    public DateTime? RevokedAtUtc { get; private set; }
    public string? ReplacedByTokenHash { get; private set; }  // trilha de rotação

    public bool IsActive => RevokedAtUtc is null && DateTime.UtcNow < ExpiresAtUtc;

    public void Revoke(string? replacedByTokenHash = null)
    {
        RevokedAtUtc = DateTime.UtcNow;
        ReplacedByTokenHash = replacedByTokenHash;
    }
}
```

O banco guarda apenas o **hash** do token. Se o banco vazar, os refresh tokens em si não são recuperáveis. A consulta no login/refresh hasheia o valor recebido e compara com o hash armazenado.

3.2. Contratos (Repositories + Unit of Work)

```
public interface IRefreshTokenRepository
{
    Task<RefreshToken?> GetByTokenHashAsync(string tokenHash, CancellationToken ct = default);
    Task AddAsync(RefreshToken refreshToken, CancellationToken ct = default);
}

public interface IUnitOfWork
{
    IUserRepository Users { get; }
    IRefreshTokenRepository RefreshTokens { get; }
    Task<int> SaveChangesAsync(CancellationToken ct = default);
}
```

4. Camada de Aplicação (Application)

Orquestra os casos de uso. Não conhece EF Core nem HTTP — depende apenas de abstrações do Domain e de interfaces próprias.

4.1. ITokenService

```
public interface ITokenService
{
    (string Token, DateTime ExpiresAtUtc) GenerateAccessToken(User user);
    (string RawToken, string TokenHash, DateTime ExpiresAtUtc) GenerateRefreshToken();
    string HashRefreshToken(string rawToken);
}
```

4.2. AuthResult — separando corpo e cookie

O serviço devolve um **AuthResult**: o `AuthResponse` (corpo HTTP, com o access token) e o **refresh token bruto**, que a API grava no cookie. O refresh token **nunca** aparece no corpo da resposta.

```
public record AuthResult(
    AuthResponse Response,        // vai no corpo (access token + dados do usuário)
    string RefreshToken,          // vai no cookie HTTP-only
    DateTime RefreshTokenExpiresAtUtc);
```

4.3. AuthService — emissão e rotação

```
// Emite o par de tokens e persiste o hash do refresh (usado em Register e Login)
private async Task<AuthResult> IssueTokensAsync(User user, CancellationToken ct)
{
    var (accessToken, accessExpires) = tokenService.GenerateAccessToken(user);
    var (rawRefresh, refreshHash, refreshExpires) = tokenService.GenerateRefreshToken();
    await unitOfWork.RefreshTokens.AddAsync(
        new RefreshToken(user.Id, refreshHash, refreshExpires), ct);
    await unitOfWork.SaveChangesAsync(ct);
    var response = new AuthResponse(accessToken, accessExpires, mapper.Map<UserDto>(user));
    return new AuthResult(response, rawRefresh, refreshExpires);
}

public async Task<AuthResult> RefreshAsync(string refreshToken, CancellationToken ct)
{
    if (string.IsNullOrWhiteSpace(refreshToken))
        throw new UnauthorizedException("Refresh token ausente.");

    var hash = tokenService.HashRefreshToken(refreshToken);
    var stored = await unitOfWork.RefreshTokens.GetByTokenHashAsync(hash, ct);
    if (stored is null || !stored.IsActive)
        throw new UnauthorizedException("Refresh token inválido ou expirado.");

    var user = await unitOfWork.Users.GetByIdAsync(stored.UserId, ct)
        ?? throw new UnauthorizedException("Usuário não encontrado.");

    // ROTAÇÃO: revoga o antigo, emite o novo, registra o vínculo
    var (accessToken, accessExpires) = tokenService.GenerateAccessToken(user);
    var (rawRefresh, refreshHash, refreshExpires) = tokenService.GenerateRefreshToken();
    stored.Revoke(replacedByTokenHash: refreshHash);
    await unitOfWork.RefreshTokens.AddAsync(
        new RefreshToken(user.Id, refreshHash, refreshExpires), ct);
    await unitOfWork.SaveChangesAsync(ct);

    var response = new AuthResponse(accessToken, accessExpires, mapper.Map<UserDto>(user));
    return new AuthResult(response, rawRefresh, refreshExpires);
}
```

5. Camada de Infraestrutura (Infraestructure)

5.1. TokenService — geração segura

```
public (string RawToken, string TokenHash, DateTime ExpiresAtUtc) GenerateRefreshToken()
{
    // 256 bits de entropia criptográfica, Base64 URL-safe
    var bytes = RandomNumberGenerator.GetBytes(32);
    var rawToken = Convert.ToBase64String(bytes)
        .Replace("+", "-").Replace("/", "_").TrimEnd('=');
    var expiresAt = DateTime.UtcNow.AddDays(_settings.RefreshTokenExpirationDays);
    return (rawToken, HashRefreshToken(rawToken), expiresAt);
}

public string HashRefreshToken(string rawToken)
{
    var hash = SHA256.HashData(Encoding.UTF8.GetBytes(rawToken));
    return Convert.ToHexString(hash);
}
```

RandomNumberGenerator (CSPRNG) garante que o token seja imprevisível. SHA-256 é determinístico — por isso serve para consulta indexada (diferente do BCrypt usado em senhas, que é salgado e não permite busca).

5.2. Mapeamento do refresh token (Fluent API)

```
builder.ToTable("refresh_tokens");
builder.HasKey(t => t.Id);
builder.Property(t => t.TokenHash).IsRequired().HasMaxLength(128);
builder.HasIndex(t => t.TokenHash).IsUnique();   // consulta O(log n) + unicidade
builder.HasIndex(t => t.UserId);
builder.HasOne<User>().WithMany().HasForeignKey(t => t.UserId)
       .OnDelete(DeleteBehavior.Cascade);
```

5.3. Configuração (appsettings.json)

```
"Jwt": {
  "Issuer": "AppExampleKafka.API",
  "Audience": "AppExampleKafka.Client",
  "SecretKey": "<chave-forte-de-32+-bytes-fora-do-repositorio>",
  "ExpirationMinutes": 60,          // access token
  "RefreshTokenExpirationDays": 7   // refresh token
}
```

6. Camada de API — Controller e Cookie

O controller é a única camada que lida com HTTP. É aqui que o refresh token é gravado/lido do cookie; o serviço nunca toca em `HttpContext`.

6.1. Configuração do cookie

```
private CookieOptions BuildCookieOptions() => new()
{
    HttpOnly = true,                // inacessível a JavaScript (mitiga XSS)
    Secure = Request.IsHttps,       // só sob HTTPS (true em produção)
    SameSite = SameSiteMode.Strict,  // mitiga CSRF
    IsEssential = true,
    Path = "/"
};

private void SetRefreshCookie(AuthResult result)
{
    var options = BuildCookieOptions();
    options.Expires = result.RefreshTokenExpiresAtUtc;
    Response.Cookies.Append("refreshToken", result.RefreshToken, options);
}
```

6.2. Endpoints

```
[HttpPost("refresh")] [AllowAnonymous]
public async Task<IActionResult> Refresh(CancellationToken ct)
{
    var refreshToken = Request.Cookies["refreshToken"];
    var result = await authService.RefreshAsync(refreshToken ?? "", ct);
    SetRefreshCookie(result);      // grava o token rotacionado
    return Ok(result.Response);    // devolve o novo access token
}

[HttpPost("logout")] [Authorize]
public async Task<IActionResult> Logout(CancellationToken ct)
{
    var refreshToken = Request.Cookies["refreshToken"];
    await authService.LogoutAsync(refreshToken, ct);       // revoga no banco
    Response.Cookies.Delete("refreshToken", BuildCookieOptions());
    return NoContent();
}
```

6.3. Tabela de endpoints

Método / Rota	Auth	Descrição
`POST /api/auth/register`	Anônimo	Cria usuário (role User) + emite tokens
`POST /api/auth/login`	Anônimo	Autentica + emite tokens
`POST /api/auth/refresh`	Cookie	Rotaciona refresh e renova access token
`POST /api/auth/logout`	Bearer	Revoga refresh e apaga o cookie
`GET /api/auth/me`	Bearer	Dados do usuário autenticado
`GET /api/auth/admin-only`	Bearer+Admin	Restrito à role Admin

7. Fluxo Completo de Requisição

7.1. Login / Register

```
Cliente ■■POST /login {email,senha}■■■ AuthController
   AuthController ■■ AuthService.LoginAsync
       ■■ FluentValidation valida a entrada
       ■■ UnitOfWork.Users.GetByEmailAsync
       ■■ PasswordHasher.Verify (BCrypt)
       ■■ TokenService.GenerateAccessToken   → JWT (60 min)
       ■■ TokenService.GenerateRefreshToken  → token bruto + hash (7 dias)
       ■■ UnitOfWork: grava hash + SaveChanges
   AuthController grava cookie HTTP-only (refresh) e retorna JSON (access)
Cliente ■■■ 200 OK + Set-Cookie: refreshToken=...; HttpOnly; Secure; SameSite=Strict
```

7.2. Renovação silenciosa (quando o access token expira)

```
Cliente ■■POST /refresh (cookie enviado automaticamente)■■■ AuthController
   AuthService.RefreshAsync
       ■■ hasheia o token do cookie e busca no banco
       ■■ valida IsActive (não revogado e não expirado)
       ■■ REVOGA o token atual (Revoke + ReplacedByTokenHash)
       ■■ emite novo access + novo refresh
       ■■ SaveChanges
   AuthController grava o NOVO cookie e retorna o NOVO access token
Cliente ■■■ 200 OK + novo Set-Cookie
```

7.3. Tratamento centralizado de erros

O **ExceptionHandlingMiddleware** converte exceções de domínio em respostas HTTP: BadRequest→400, Unauthorized→401, Conflict→409; qualquer outra exceção vira 500 (logada). O corpo segue o formato ProblemDetails.

8. Como Executar e Testar

8.1. Pré-requisitos e execução

```
# PostgreSQL já rodando (container), mapeando 5437 -> 5432
# String de conexão em appsettings.json (User Id=postgres)

dotnet build
dotnet run --project AppExampleKafka.API
# As migrations são aplicadas automaticamente na inicialização
#   (Database.Migrate() no Program.cs) — cria as tabelas
#   users e refresh_tokens.
```

8.2. Exercitando o fluxo com cURL (mantendo cookies)

```
# -c/-b guardam e reenviam o cookie HTTP-only automaticamente
curl -c cj.txt -X POST localhost:5080/api/auth/register \
  -H 'Content-Type: application/json' \
  -d '{"name":"Ana","email":"ana@x.com","password":"secret123"}'

# Renova o access token usando o cookie de refresh
curl -b cj.txt -c cj.txt -X POST localhost:5080/api/auth/refresh

# Logout (revoga o refresh e apaga o cookie)
curl -b cj.txt -X POST localhost:5080/api/auth/logout \
  -H 'Authorization: Bearer <access_token>'
```

8.3. Resultado da validação ponta a ponta

Cenário	Esperado	Obtido
Register	200 + Set-Cookie HttpOnly	**OK**
Cookie contém HttpOnly + SameSite=Strict	Sim	**OK**
Login	200 + cookie	**OK**
Refresh com cookie	200 + novo cookie (rotação)	**OK**
Refresh sem cookie	401	**OK**
Logout	204 + cookie apagado	**OK**
Refresh após logout (revogado)	401	**OK**

9. Notas e Próximos Passos

- **AutoMapper 14.0.0** é usado deliberadamente (v15+ exige licença comercial). O aviso NU1903 refere-se a recursão profunda e não se aplica ao mapeamento plano User→UserDto.
- **SecretKey** deve ser substituída por um segredo forte, fora do repositório (variável de ambiente ou secret manager) em produção.
- Em produção, com HTTPS, `Secure` torna-se `true` automaticamente (`Request.IsHttps`).
- Registro sempre cria role **User**. Para um Admin, use um seed inicial ou o método de domínio `User.ChangeRole` em um fluxo administrativo.
- Evoluções possíveis: limpeza periódica de refresh tokens expirados; detecção de reuso (revogar toda a cadeia ao detectar um token já rotacionado); endpoint para revogar todas as sessões de um usuário.